# Unity 3D에서 위치 기반 동역학과 내부 형상 보존 제약을 활용한 하이브리드 CPU-GPU 절단 시뮬레이션

Hybrid CPU-GPU Cutting Simulation Using Position-Based Dynamics and Internal Shape-Preserving Constraints in Unity 3D

Lyudmila Khan · Yoo-Joo Choi · Min Hong

소속: 순천향대학교 소프트웨어융합학과; 서울미디어대학원대학교 AI소프트웨어공학과; 순천향대학교 컴퓨터소프트웨어공학과. 교신저자: Min Hong.

Electronics 2026, 15, 3796 · DOI: 10.3390/electronics15173796

접수: 2026년 6월 30일 · 수정: 8월 1일 · 게재 승인: 8월 20일 · 출판: 8월 24일. 학술 편집자: Javid Taheri.

번역 안내: 첨부 논문의 본문, 알고리즘, 그림 설명, 표, 부록 및 말미 선언을 한국어로 번역했다. 원문 페이지는 삽입하지 않았다. 수식과 그림은 원본을 유지했으며, 그림 안의 주요 영문 용어는 한국어 설명으로 풀이했다. 참고문헌 서지 정보는 대조와 검색을 위해 원문으로 보존했다. 원문에 제시된 수치와 표현은 임의로 수정하지 않았다.

## 초록

절단 시뮬레이션은 수술 시뮬레이터의 핵심 구성요소로, 실시간 성능, 수치적 안정성, 물리적으로 그럴듯한 변형이 요구된다. 절단은 충돌 검출, 동적 위상 변경, 계산 집약적인 물리 시뮬레이션을 결합하므로 이러한 특성을 달성하기 어렵다. 이 연구에서는 변형 가능한 물체를 위치 기반 동역학(Position-Based Dynamics, PBD)으로 모델링하고, 삼각형 표면 메시에 적용한 내부 형상 보존 제약(Internal Shape-Preserving Constraints, ISPC)으로 체적 거동을 근사한다.

주요 기여는 Unity 3D의 하이브리드 CPU-GPU 절단 아키텍처이다. 계산 비용이 큰 삼각형-삼각형 교차 검사를 컴퓨트 셰이더로 병렬화하고, GPU에 상주하는 기하 버퍼에서 직접 표면을 렌더링하며, CPU-GPU 동기화 오버헤드를 최소화한다. 변형과 충돌 처리만 병렬화했던 이전 구현에 비해, 제안 프레임워크는 절단의 교차 검출 단계도 가속한다.

복잡도가 증가하는 8개 삼각형 메시(삼각형 20~30,000개)에 대해 단일 절단 및 다중 절단 실험을 수행했다. 8개 중 7개 모델은 상호작용 가능한 프레임률을 유지했다. 반복적인 자유 절단에서도 절단 단계의 평균 비용은 0.5ms 미만으로 정상 상태 비용에 거의 기여하지 않았으며, 프레임 시간은 주로 변형과 조각 간 충돌 처리에 좌우되었다. 이전 연구 대비 평균 프레임률은 슬라이싱 기반 방법보다 최대 677%, CPU 기반 ISPC 구현보다 최대 395% 향상되었다. 기하 복잡도가 낮은 모델은 초기 체적 대비 0.4% 이내의 차이로 체적을 유지했다.

두 가지 한계가 관찰되었다. 고해상도 Armadillo 모델은 평균 프레임 비용이 변형과 충돌에 지배되었고, 드물게 발생하는 절단 마무리 프레임에서 큰 최악의 지연이 생겨 상호작용 가능한 프레임률에 미치지 못했다. Octopus 모델은 체적 보존 성능이 크게 떨어져 초기 체적의 73.4%에서 안정화되었다. 결과적으로 하이브리드 방법은 절단 시뮬레이션의 효율을 크게 개선하지만, 크고 기하학적으로 복잡한 모델로의 확장은 향후 과제

로 남는다.

주제어: 절단 시뮬레이션, 위치 기반 동역학, 수술 시뮬레이터, 병렬 컴퓨팅, 내부 형상 보존 제약.

# 1. 서론

절단 과정의 시뮬레이션은 기하학적 변경과 복잡한 물리적 반응을 모두 요구하므로 컴퓨터 그래픽스와 물리 기반 모델링의 근본적인 과제이다. 절단 시뮬레이션은 애니메이션, 디지털 모델링, 생체역학, 의료 교육 등 여러 분야에 응용된다. 가상현실(VR)과 증강현실(AR)의 빠른 발전으로 시뮬레이션 기반 훈련은 특히 의학에서 점점 중요해지고 있다. 기존 훈련과 비교하면 VR 훈련은 복강경 수술에서 수술 시간과 기구·조직 취급을 개선하고 오류를 줄일 수 있다[1]. VR 기반 신경해부학 교육이 기존 교육과 비슷한 학습 성과를 달성하면서 참여도, 즐거움, 학습 동기를 더 높인다는 근거도 있다[2]. VR·AR 시뮬레이터의 가장 큰 장점은 안전하고 반복 가능한 연습을 제공하고 절차에 대한 이해를 높인다는 점일 것이다[3,4].

이런 시스템에서 가장 어려운 부분 중 하나는 절단 연산의 시뮬레이션이다. 절단에는 보통 비선형 변형 거동을 처리하면서 동시에 메시 위상을 갱신하는 작업이 필요하다[5]. 전통적인 접근법은 흔히 메시 기반 표현, 특히 사면체 메시를 사용한다. 이는 정확한 체적 모델링을 제공하지만 계산 비용이 클 수 있다[6]. 이러한 방법은 유한요소법(FEM)과 결합되는 경우가 많아, 매우 사실적인 재료 변형을 구현하는 대신 계산 복잡도가 더 높아진다[7].

이러한 한계를 극복하기 위해 무메시 또는 하이브리드 표현에 기반한 대안이 제안되었다. 이 기법들은 허용 가능한 물리적 정확도를 유지하면서 위상 관리를 단순화한다. 예를 들어 [8]은 형상 함수 갱신과 균열 전파를 효율적으로 처리하는 무메시 절단 방법을 소개한다. [9]는 고해상도 시각 모델과 효율적인 저해상도 역학 모델을 결합한 하이브리드 프레임워크를 제시한다. 최근 연구는 멀티코어 CPU와 GPU 병렬 구현[10,11], 충돌 검출과 기하 질의를 가속하는 바운딩 볼륨 계층구조[12]를 통해 계산 문제를 더 다루었다.

이 연구에서는 표면 기반 메시 표현과 ISPC[13]를 결합한 절단 시뮬레이션 프레임워크를 제시한다. 변형은 PBD로 시뮬레이션하고, ISPC는 절단 과정 전반에서 물체의 구조적 일체성을 유지한다. 이전 연구와 달리 본 연구의 주요 목표는 GPU에서 절단 연산을 병렬화하여 계산 효율을 개선하는 것이다.

또 다른 주요 성능 병목은 빈번한 CPU-GPU 데이터 전송으로 인한 동기화 오버헤드와 실행 정체이다. 이를 해결하기 위해 양방향 데이터 전송을 최소화하도록 시스템 아키텍처를 재설계하여 전체 실행 성능을 개선한다. 논문의 구성은 다음과 같다. 2절에서는 방법론과 핵심 기법을 설명하고, 3절에서는 실험 데이터를 제시하며, 4절에서는 결과와 한계, 향후 연구를 논의한다.

# 2. 재료 및 방법

## 2.1. 계산 모델

제안 방법은 PBD를 핵심 시뮬레이션 프레임워크로 사용한다. PBD에서는 물체를 제약으로 연결된 이산 점들의 집합으로 표현한다. 제약은 점에 힘을 가하는 대신 위치를 직접 조절한다[14]. 이 정식화는 수치적 안정성, 강건성, 계산 효율성 때문에 실시간 응용에 특히 적합하다[15]. [16]은 햅틱 피드백과 함께 실시간 담낭절제술을 시뮬레이션하는 데 PBD를 적용했다. [17]은 대화형 AR 의료 교육에서 PBD를 효율적으로 사용할 수 있음을 보였다. [18]은 형상 매칭 제약을 갖는 그래프 기반 PBD로 연조직에 실시간이고 유연한 점진적·불연속 절단을 구현했다. [19]는 실제 환경에서 거의 완벽한 절단을 달성하기 위한 수술 로봇 훈련에 PBD 시뮬레이션을 적용했다.

PBD의 제약을 풀어 위치 보정량 Δp를 얻으며, 제약 함수는 C(p+Δp)=0으로 정의할 수 있다. 이를 기울기 벡터 ∇p C(p)로 근사하면 식 (1)이 된다.

$$C(p+\Delta p) \approx C(p) + \nabla_p C(p) \cdot \Delta p = 0 \tag{1}$$

위치 보정량 Δp가 기울기 방향과 나란하도록 스칼라 승수 λ를 도입한다.

$$\Delta p = \lambda \nabla_p C(p) \tag{2}$$

식 (2)를 식 (1)에 대입하면 λ를 다음과 같이 구할 수 있다.

$$\lambda = -\frac{C(p)}{|\nabla_p C(p)|^2} \tag{3}$$

각 정점의 변위는 다음과 같이 계산한다.

$$\Delta p_i = -\frac{w_i C(p)}{\sum_j w_j |\nabla_{p_j} C(p)|^2} \nabla_{p_i} C(p) \tag{4}$$

여기서 wi는 정점 pi의 역질량이다.

시뮬레이션에는 거리 제약과 굽힘 제약의 두 종류가 포함된다. 거리 제약은 점 쌍을 연결하여 두 점의 거리를 조절하고 과도한 늘어남이나 압축을 방지한다. 거리 제약 함수는 Cd(p1,p2)=|p1−p2|−d로 유도되며, d는 p1과 p2 사이의 기준 거리이다. 각 정점의 위치 보정량은 다음과 같다.

$$\Delta p_1 = -\frac{w_1}{w_1 + w_2}(|p_1 - p_2| - d)\frac{p_1 - p_2}{|p_1 - p_2|} \tag{5}$$

$$\Delta p_2 = \frac{w_2}{w_1 + w_2}(|p_1 - p_2| - d)\frac{p_1 - p_2}{|p_1 - p_2|} \tag{6}$$

굽힘 제약은 인접 표면 요소 사이의 각도를 보존하여 과도한 펼쳐짐과 뒤집힘을 줄인다. 굽힘 제약은 인접한 두 삼각형 (p1,p2,p3)와 (p1,p2,p4)로 구성한다. 제약 함수는 다음과 같다.

$$\begin{aligned} C_b(p_1, p_2, p_3, p_4) &= \arccos(n_1 \cdot n_2) - \varphi_0 \\ &= \arccos\left(\frac{(p_2-p_1)\times(p_3-p_1)}{|(p_2-p_1)|\times|(p_3-p_1)|} \cdot \frac{(p_2-p_1)\times(p_4-p_1)}{|(p_2-p_1)|\times|(p_4-p_1)|}\right) - \varphi_0 \end{aligned} \tag{7}$$

여기서 n1과 n2는 각 삼각형의 법선 벡터이고, φ0는 두 삼각형 사이의 기준 각도이다. 삼각형 점에 대한 미분은 다음과 같이 계산한다.

$$\nabla_{p_4} C = -\frac{1}{\sqrt{1-d^2}}\left(\left(\frac{\partial n_2}{\partial p_4}\right)^T n_1\right) \tag{8}$$

$$\nabla_{p_3} C = -\frac{1}{\sqrt{1-d^2}}\left(\left(\frac{\partial n_1}{\partial p_3}\right)^T n_2\right) \tag{9}$$

$$\nabla_{p_2} C = -\frac{1}{\sqrt{1-d^2}}\left(\left(\frac{\partial n_1}{\partial p_2}\right)^T n_2 + \left(\frac{\partial n_2}{\partial p_2}\right)^T n_1\right) \tag{10}$$

$$\nabla_{p_1} C = -\nabla_{p_2} C - \nabla_{p_3} C - \nabla_{p_4} C \tag{11}$$

이후 정규화된 외적의 기울기를 각 정점에 대해 계산한다.

$$q_4 = \frac{p_2 \times n_1 + (n_2 \times p_2)d}{|p_2 \times p_4|} \tag{12}$$

$$q_3 = \frac{p_2 \times n_2 + (n_1 \times p_2)d}{|p_2 \times p_3|} \tag{13}$$

$$q_2 = -\frac{p_3 \times n_2 + (n_1 \times p_3)d}{|p_2 \times p_3|} - \frac{p_4 \times n_1 + (n_2 \times p_4)d}{|p_2 \times p_4|} \tag{14}$$

$$q_1 = -q_2 - q_3 - q_4 \tag{15}$$

최종 위치 보정량은 다음과 같다.

$$\Delta p_i = -\frac{w_i \sqrt{1-d^2}(\arccos(d) - \varphi_0)}{\sum_j w_j |q_j|^2} q_i \tag{16}$$

이러한 위치 조작으로 사실적인 변형과 형상 복원이 가능하다. 본 연구에서는 시간 적분에 반암시적 오일러 방법을 사용하고, 제약 투영은 Jacobi 방식의 솔버로 구현한다.

그러나 기하 구조가 복잡한 물체에서는 거리와 굽힘 제약만으로 큰 변형에서 복원하기에 충분하지 않다. 따라서 물체 내부의 점들을 연결하여 전체 구조의 일관성과 형상을 유지하는 특수한 거리 제약인 ISPC를 도입한다. 수학적으로 ISPC는 거리 제약과 동일하다. 이러한 연결들이 함께 내부 구조를 이루어 표면 메시에 실질적인 체적 지지를 제공한다. 이전 연구[13,20]는 ISPC가 과도한 변형을 제한하고 원래 형상의 복원을 개선하며 결과적으로 전체 체적을 보존함을 보였다.

ISPC는 정점 vi에서 정점 법선의 반대 방향 −ni를 따라 고정 길이의 유한 광선을 발사하여 생성한다. 광선은 현재 유효한 모든 삼각형과 검사하되, vi에 접한 삼각형과 바깥쪽 법선이 광선 방향에 반대인 삼각형은 제외한다. 가장 가까이 교차한 삼각형을 선택한다. 제약의 끝점 vj는 해당 삼각형의 꼭짓점 중 교차점에 가장 가까운 것으로 정하고, 생성 시점의 거리 l0=|vi−vj|를 기준 길이로 초기화한다.

## 2.2. GPU 가속 과정

절단 시뮬레이션의 계산 성능을 높이기 위해 Unity의 컴퓨트 셰이더[21]를 사용한다. 컴퓨트 셰이더는 HLSL로 작성하는 범용 GPU 프로그램으로, 높은 병렬성을 제공한다. 계산 집약적인 작업을 CPU에서 GPU로 옮기면 수천 개 스레드가 같은 연산을 동시에 수행할 수 있다. 정점·프래그먼트 셰이더와 달리 컴퓨트 셰이더는 그래픽 렌더링 파이프라인과 독립적이어서 임의의 병렬 계산에 사용할 수 있다. 기능은 커널로 구성되며, 미리 지정된 스레드 그룹 크기에 따라 실행된다.

구현에서는 PBD 제약 풀이, 삼각형-삼각형 교차 검출, ISPC 갱신, 충돌 검출과 반응을 모두 컴퓨트 셰이더로 병렬화한다. 제약을 푸는 동안 경쟁 상태를 방지하기 위해 정점별 버퍼에 위치 보정량을 원자적으로 누적한다.

이전 연구와 비교하여, 계산 비용이 큰 삼각형-삼각형 교차 검사를 GPU로 옮겨 CPU에서 순차 검사하는 대신 모든 삼각형 쌍을 동시에 처리할 수 있다. 검출한 교차는 IntersectBuf 컴퓨트 버퍼에 누적하고 이후 처리를 위해 CPU로 전송한다. 나머지 절단 단계는 CPU에서 실행한다. 이 단계는 메시 인접성 정보에 의존하며, GPU에서 효율적으로 지원하지 않는 동적 딕셔너리 형태의 자료구조가 필요하기 때문이다.

절단 연산이 끝나면 새 정점과 삼각형을 GPU 컴퓨트 버퍼에 업로드한다. 모든 기하 버퍼는 초기화 때 불러온 메시 크기의 고정 배수로 한 번 할당한다. 전송 오버헤드를 더 줄이기 위해 표면은 이 컴퓨트 버퍼에서 직접 렌더링한다. 시뮬레이션과 렌더링이 같은 기하 데이터를 사용하므로 CPU-GPU 동기화와 메모리 전송이 최소화되어 전체 시뮬레이션 성능이 개선된다.

## 2.3. 충돌 처리

제안 방법은 정점-삼각형 자기 충돌을 구현한다. 검사를 가속하기 위해 삼각형의 공간 해시[22]를 사용하고 매 프레임 재구축한다. 각 삼각형은 주변 3×3×3 셀에서 후보를 모으며, 정점-삼각형 검사로 교차를 검출한다. 정점을 공유하는 삼각형 같은 위상적 이웃은 검사에서 제외한다. 위치 보정량도 원자적으로 누적한다.

절단 중 생성된 정점은 절단이 끝나고 안정화될 때까지 충돌 검사에서 제외한다. 그렇지 않으면 같은 위치에 있는 절단 경계의 중복 정점 때문에 폭발적인 움직임이 생길 수 있다.

## 2.4. 절단 시뮬레이션

절단 절차는 이전 연구[20]의 네 주요 단계와 일치한다. 절단 도구와 물체의 교차 검출, 영향을 받은 삼각형의 세분화, 메시 틈 채우기, 시뮬레이션 제약 갱신이다. 절단 알고리즘을 알고리즘 1에 요약한다.

절단 도구와 표면 메시의 충돌을 검출하기 위해 칼날을 두 끝점으로 정의한 선분으로 이산화한다. 현재 프레임과 이전 프레임의 두 끝점 위치로 쓸기 평면을 동적으로 구성하여 시간에 따른 칼날의 움직임을 표현한다. 쓸기 평면은 두 삼각형으로 분해하고, 부호 있는 사면체 체적 검사로 메시 삼각형과의 교차를 확인한다.

현재 구현에서는 정확히 쓸기 평면 위에 놓인 점과, 쓸기 사각형의 모서리 또는 정점을 지나는 선분이 관통 검사에 통과하지 못한다. 따라서 동일 평면 위의 모서리와 관통하지 않고 접촉만 하는 경우는 교차하지 않은 것으로 보고한다. 절단이 정점 근처를 지나면 지나치게 작거나 가느다란 삼각형을 피하기 위해 교차점을 정점으로 스냅한다. 각 삼각형에서 교차된 모서리 및 정점의 수와 같은 정보를 기록한다. 모든 삼각형-삼각형 교차 검사

는 Unity 컴퓨트 셰이더를 이용해 GPU에서 병렬로 수행한다.

검사가 끝나면 교차 데이터를 CPU로 전송하고 결과에 따라 4상태 머신을 갱신한다. 상태는 '절단 시작', '절단 중', '절단 종료', '절단 없음'이다. 쓸기 평면의 앞쪽·뒤쪽 모서리에 교차가 발생했는지에 따라 상태를 갱신한다. 앞쪽 모서리는 도구 끝점의 현재 위치를 연결한 선분이고, 뒤쪽 모서리는 이전 프레임의 끝점 위치를 연결한 선분이다.

앞쪽 모서리가 표면 삼각형과 교차하면 '절단 시작'을 검출한다. 앞쪽과 뒤쪽 모서리가 모두 교차하면 '절단 중' 상태가 된다. 앞쪽 모서리는 교차하지 않고 뒤쪽 모서리만 교차하면 '절단 종료'로 정한다. 어느 모서리도 교차하지 않으면 '절단 없음'이다. 상태 전이는 현재 단계에서 절단을 수행할지, 제약 재구축과 데이터 초기화로 절단을 마무리할지를 결정한다.

교차 정보로 각 삼각형의 교차 유형을 분류한다. 그림 1은 구현한 세 유형을 보여준다. (a) 두 모서리가 교차한 삼각형은 새 삼각형 3개로 나눈다. (b) 모서리 하나와 정점 하나가 교차한 삼각형은 2개로 나눈다. (c) 정점 두 개가 교차하면 공유 모서리를 복제하여 절단 경로를 따라 인접 삼각형을 분리한다. 각 연산은 절단 양쪽의 열린 경계 모서리를 기록하고, 인접성 정보를 갱신하며, 제거하거나 생성할 제약을 큐에 넣는다.

## 알고리즘 1. 점진적 절단 (매 프레임 한 번 실행)

입력: 절단 도구 끝점 - 현재 p0, p1 및 이전 p0′, p1′; 기하 데이터 - 정점 목록 V, 삼각형 목록 T, 인접성; 제약 - 거리, 굽힘, ISPC; 상태 s∈{NO-CUT, CUT_START, CUTTING, CUT_END}.

출력: 갱신된 V, T, 인접성, 거리·굽힘·ISPC 제약, s.

1. [CPU] {p0,p1,p0′,p1′}로 쓸기 사각형을 구성하고 두 삼각형 Q로 나눈다.

2. [GPU] T의 각 삼각형 t를 병렬로 처리한다.

3. 부호 있는 사면체 체적 검사로 t와 Q를 비교한다.

4. 교차 모서리·정점의 수와 교차점을 IntersectBuf에 기록한다.

5. [동기화] IntersectBuf의 교차 요약을 GPU에서 CPU로 읽어온다.

6. [CPU] 요약으로 상태 머신 s를 갱신한다.

7. s가 CUT_START 또는 CUTTING이면 절단을 수행한다.

8. [CPU] 교차한 각 삼각형에 대해:

9. 교차 유형을 분류한다(그림 1).

10. 세분화하고 원자 카운터를 통해 미리 할당한 버퍼에 새 V, T를 추가한다.

11. 절단 양쪽의 열린 경계 모서리를 기록하고 인접성을 갱신한다.

12. 오래된 제약의 제거 및 새 제약을 추가할 모서리를 큐에 넣는다.

13. s가 CUT_END이면 마무리한다.

14. [CPU] 절단 틈을 채운다: 이어 클리핑 삼각분할 + 델로네 세분화.

15. 카운터를 통해 패치의 V, T를 버퍼에 추가한다.

16. [CPU] 큐에 넣은 제약을 제거한다.

17. [CPU] 새 삼각형의 거리·굽힘 제약을 추가한다.

18. [CPU] ISPC를 추가한다: 공간 격자 순회로 내부 점 쌍을 찾는다.

19. [동기화] 추가한 V/T/제약을 GPU 버퍼로 보내고 개수를 갱신한다.

20. [GPU] 표면 렌더링이 기하 버퍼를 직접 읽는다.

표면 메시에서 작동하므로 절단은 기하 구조에 열린 부분을 만든다. 수밀한 표면을 유지하려면 이를 닫아야 한다. 기존 경계 정점을 재사용하는 이어 클리핑 삼각분할[23]로 열린 부분을 채운다. 메시 품질과 원래 해상도를 유지하기 위해 생성된 삼각형을 델로네 방식의 분할 접근법[24]으로 추가 세분화한다.

각 절단 연산은 메시 기하를 변경하므로 대응하는 GPU 자료구조도 갱신해야 한다. 절단마다 컴퓨트 버퍼를 재할당·재초기화하지 않도록 모든 기하 버퍼에 추가 용량을 미리 할당하고, 현재 유효한 정점·삼각형·제약 수를 추적하는 요소 카운터를 유지한다. 새 요소는 해당 버퍼 뒤에 추가하고 카운터를 갱신한다. 비용이 큰 GPU 읽어오기를 최소화하기 위해 기하 데이터를 CPU와 GPU 양쪽에서 유지하며, 각 절단 후 변경된 요소만 동기화한다.

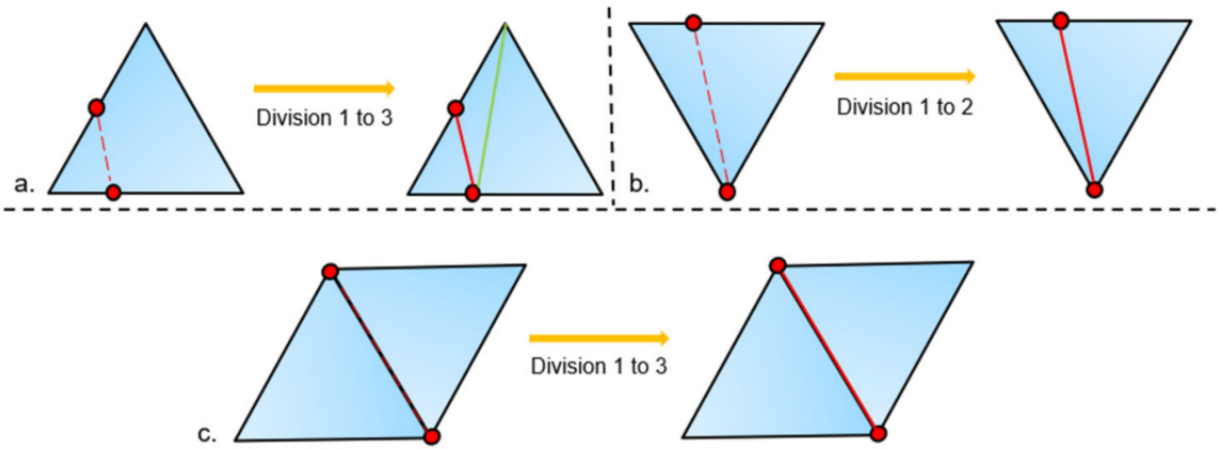



그림 1. 삼각형 세분화 방식. (a) 1개를 3개로 분할, (b) 1개를 2개로 분할, (c) 모서리 복제. 검은 실선은 기존 삼각형 모서리, 빨간 점선은 도구와 삼각형의 교차 위치, 빨간·녹색 실선은 분할로 생성한 새 모서리이다.

기하를 갱신한 뒤 변경된 메시 위상을 반영하도록 PBD 제약을 재구축한다. 제거한 요소와 연결된 제약은 폐기하고 새 정점·삼각형에는 새 제약을 생성한다. 제약 버퍼도 기하 버퍼처럼 여유 용량을 할당하여, 전체 버퍼를 재초기화하지 않고 제약을 추가할 수 있게 한다.

절단 중 ISPC는 절단 단계마다 한 번 갱신한다. ISPC 선분이 쓸기 평면과 교차하거나, 끝점이 서로 다른 절단 조각에 속하면 유효한 ISPC를 제거한다. 첫 번째 조건은 진행 중인 절단에서 제약을 끊고, 두 번째 조건은 메시 분리 후 남은 조각 간 연결을 반드시 제거한다. 현재 유효한 제약을 갖지 않은 모든 정점에 같은 내부 방향 광선 검사를 다시 수행하여 제약을 생성한다. 공간 격자와 레이 마칭을 결합해 연결 후보를 효율적으로 찾고, 모든 메시 삼각형을 탐색하는 비싼 검사를 없앤다. 원래 정점과 다른 조각에 속한 후보 삼각형은 거부하여 분리된 조각을 제약이 다시 연결하지 않도록 한다.

# 3. 결과

제안 방법은 Unity 2022.3.30f1과 C# 9.0으로 구현했다. Intel i7-10700KF CPU 3.80GHz, NVIDIA GeForce RTX 3060 Ti GPU, RAM 32GB, VRAM 8GB 시스템에서 시험했다. 절단 실험 모델의 정보는 표 1에 제시한다. 모든 실험에서 시간 간격은 0.005, 프레임당 반복 횟수는 30으로 설정했다. 거리 및 굽힘 제약의 강성은 0.5, ISPC는 0.3으로 정했다. Octopus 모델에서는 체적 보존을 높이기 위해 세 종류 제약의 강성을 모두 1로 설정했다. 엡실론 값은 $10^{-7}$~$10^{-12}$ 범위를 사용했다.

## 표 1. 절단 실험에 사용한 3D 모델 정보

| 모델 | 정점 | 삼각형 | 거리 제약 | 굽힘 제약 | ISPC |
|---|---|---|---|---|---|
| Sphere 20 | 12 | 20 | 30 | 30 | 6 |
| Sphere 80 | 42 | 80 | 120 | 120 | 21 |
| Torus | 400 | 800 | 1200 | 1200 | 200 |
| Bunny | 752 | 1500 | 2250 | 2250 | 537 |
| Octopus | 1410 | 2816 | 4224 | 4224 | 958 |
| Banana | 2798 | 5592 | 8388 | 8388 | 2262 |
| Liver | 3183 | 6362 | 9543 | 9543 | 2354 |
| Armadillo | 15002 | 30000 | 45000 | 45000 | 10710 |

### 3.1. 점진적 절단 실험

정적인 모델 집합에서 절단 실험을 수행했다. 그림 2와 3은 Sphere 20, Sphere 80, Torus, Bunny, Octopus, Banana, Liver, Armadillo의 절단과 이후 변형을 보여준다. 세분화 전략은 절단 도구의 경로를 밀접하게 따르는 정밀한 절단면을 생성했다. 절단 후에는 중력과 충돌 처리를 활성화하여 분리된 조각이 서로 및 바닥과 자연스럽게 상호작용하도록 했다. 충돌 때 큰 변형을 겪었지만 모든 모델은 ISPC 덕분에 구조적 일체성을 유지했다. 고해상도 메시의 변형이 더 컸지만 충돌 후 전체 체적을 유지하면서 원래 형상을 복원했다. 각 모델을 10회 절단하고 정량적인 성능 데이터를 얻었다.

표 2는 평균 FPS와 변형·충돌·절단 시간을 ms로 제시한다. 각 모델을 10초 구간에서 0.1초 간격으로 프로파일링했다. Unity에서는 디스패치를 비동기로 제출하므로, 단계 주변의 호스트 타이머는 실행이 아니라 명령 제출 시간을 측정한다. 단계별 실행 시간을 얻기 위해 각 단계 경계에 명시적 동기화 지점을 삽입했다. 단일 요소 버퍼를 블로킹 방식으로 읽어와 호스트 타이머를 멈추기 전에 명령 큐의 작업이 완료되도록 한다. 따라서 각 구간은 단계 시작부터 마지막 디스패치가 장치에서 완료될 때까지의 실제 경과 시간을 보고한다.

이 계측은 호스트와 장치를 직렬화한다. 일반 실행에서는 장치가 하위 단계 n을 실행하는 동안 호스트가 n+1을 제출한다. 계측할 때는 이 중첩이 제거되며 각 단계는 비어 있는 파이프라인에서 시작한다. 그러므로 여기서 보고하는 단계별 시간은 계측하지 않은 프레임에서 각 단계가 기여하는 시간의 상한이며, 단계별 수치를 합쳐도 측정한 프레임 시간이 되지 않는다.

두 종류 수치는 동일한 장면·모델·매개변수 설정의 별도 실행에서 보고한다. 프레임 시간은 계측 없이 연속한 프레임 시작 사이의 간격으로 측정하고 수직 동기화는 끈다. 단계별 세부 시간은 별도의 계측 실행에서 측정한다. 절단 시간은 위상 갱신이 실제 발생한 프레임만 대상으로 하므로, 프레임당 평균이 아니라 조건부 평균이다.

표 3은 단계별 시간의 표준편차(SD), 중앙값, P99, P95를 제시한다. Armadillo를 제외한 모든 모델에서는 절단보다 변형·충돌 시간이 지배적이다. Torus 이상의 5개 모델에서 변형과 충돌의 비중은 매우 안정적이며 변형의 영향이 조금 더 크다. 단순한 두 구 모델은 예외로, 변형의 고정 반복 오버헤드가 사실상 비어 있는 충돌 작업량보다 크다.

중간 크기 모델인 Bunny, Octopus, Banana, Liver는 매우 안정적이며 변형·충돌 단계의 P99가 중앙값의 25% 범위 안에 있다. 분포는 대체로 대칭으로, 큰 정체 없는 정상 상태 거동을 나타낸다. Armadillo는 실제로 큰 변동을 보인다. 절단의 SD는 18.949ms, 조건부 평균은 11.123ms, 최댓값은 72.419ms로 다른 단계보다 영향이 크다. 절단 중앙값은 0.047ms에 불과하므로 분포의 꼬리가 매우 두껍다. 대부분의 절단 프레임은 저렴하지만 일부 프레임은 재삼각분할과 제약 갱신 비용이 매우 크다.

30Hz의 상호작용 예산을 기준으로 평균 성능에서 Armadillo를 제외한 모든 모델이 이를 만족한다. 그림 4와 5는 8개 모델의 10초 동안 평균 FPS 변화를 보여준다. 모델별 전체 시뮬레이션·변형·충돌·절단 시간의 상세 표현은 부록 A의 그림 A1에 있다.

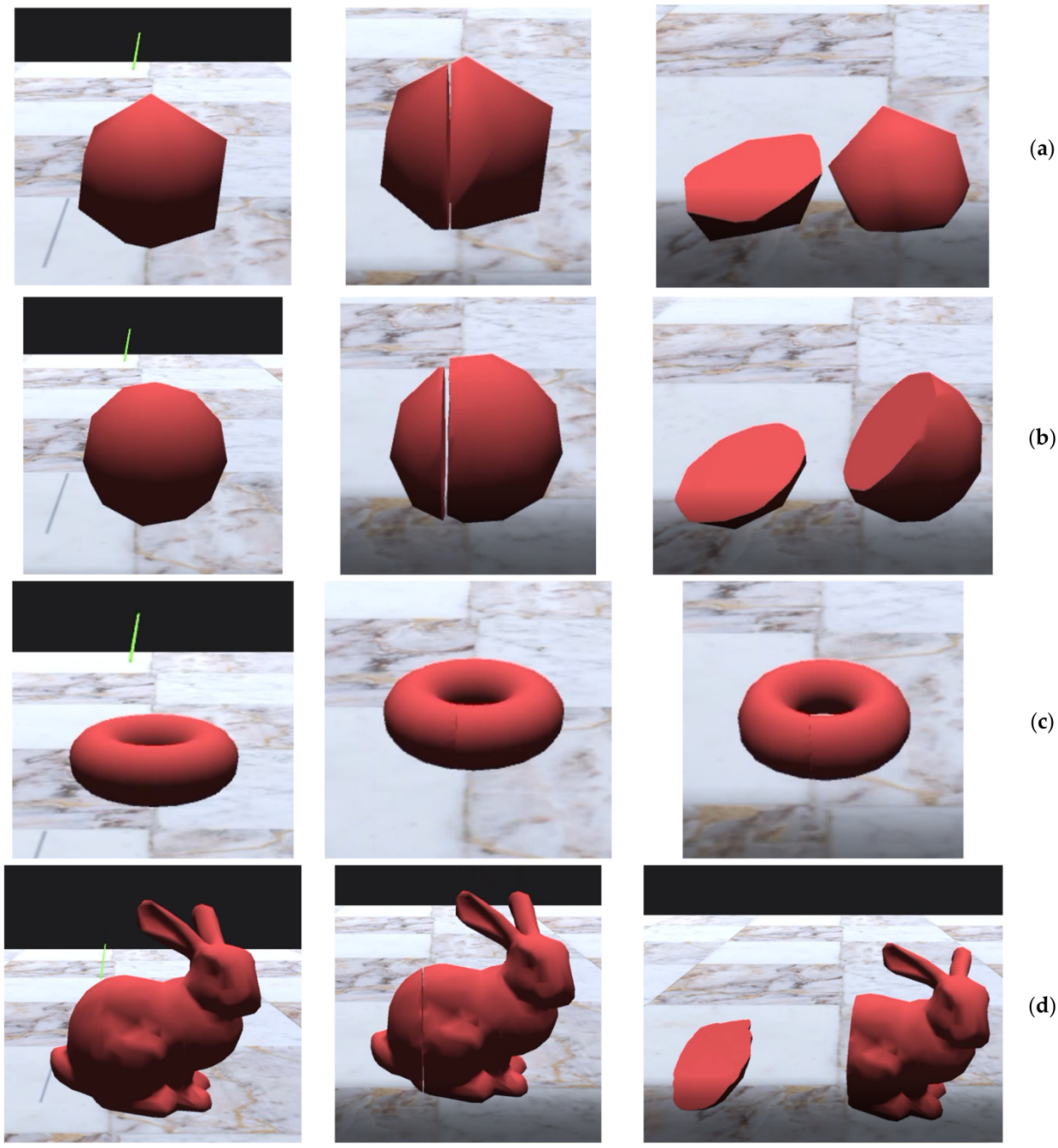



그림 2. 절단 시뮬레이션 결과. (a) Sphere 20, (b) Sphere 80, (c) Torus, (d) Bunny. 녹색 선은 절단 도구의 경로를 나타낸다.

CPU와 GPU 시간 질의를 평가했다. 프레임별 시간을 약 0.1초 간격으로 기록하고, 공통 0.1초 격자에 실행들을 정렬하여 초기 과도 구간 이후의 10초 정상 상태 구간에서 구간별로 평균했다. 절단이 발생하지 않은 프레임은 절단 통계에서 제외했다. 그림 6은 모델별 CPU·GPU 평균 단계 시간의 시계열과 히스토그램 분석을 보여준다.

Armadillo를 제외한 모든 경우에서 변형과 자기 충돌은 프레임 시간의 95% 이상을 차지했다. 절단의 중앙값 기여는 매우 작지만 분포는 오른쪽으로 크게 치우쳤다. 프레임별 최댓값은 Sphere 20의 0.283ms에서 Banana 4.976ms, Liver 7.640ms로 증가했다. 이 피크는 CPU에서 절단 틈 삼각분할과 제약 재구축을 수행하는 절단 마무리 프레임과 일치했다. Torus는 안정성의 유일한 예외로, 절단이 끝나고 분리된 조각이 지속적으로 접촉하기 시작하면 비용이 약 5.6ms에서 18ms로 거의 세 배가 되었다.

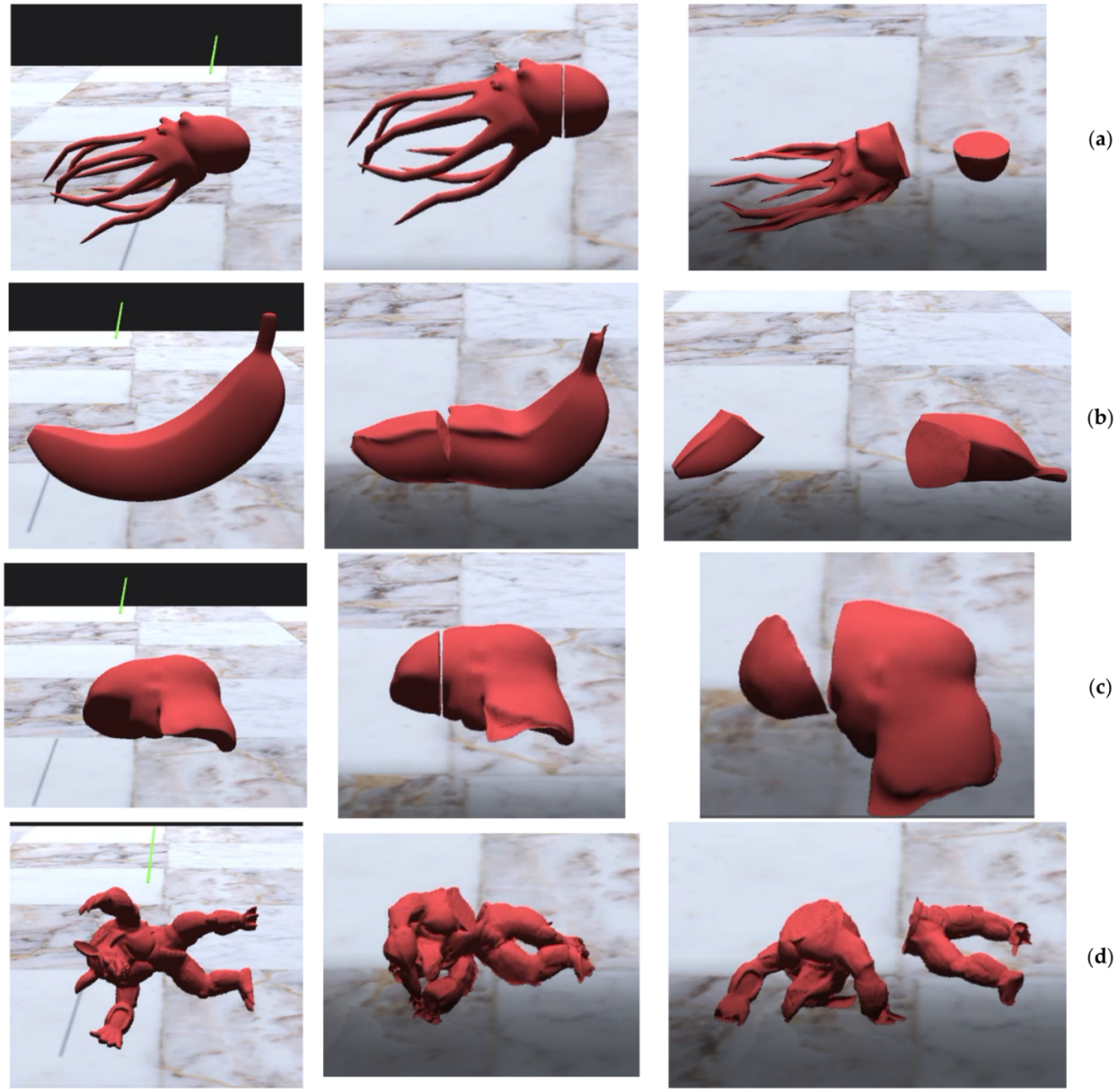



그림 3. 절단 시뮬레이션 결과. (a) Octopus, (b) Banana, (c) Liver, (d) Armadillo. 녹색 선은 절단 도구의 경로를 나타낸다.

## 표 2. GPU ISPC의 평균 FPS 및 단계별 시간

| 모델 | FPS<br>평균 (최소, 최대) | 변형 시간(ms)<br>평균 (최소, 최대) | 충돌 시간(ms)<br>평균 (최소, 최대) | 절단 시간(ms)<br>평균 (최소, 최대) |
|---|---|---|---|---|
| Sphere 20 | 481.04 (323.71, 629.86) | 0.854 (0.684, 3.901) | 0.209 (0.130, 0.938) | 0.034 (0.000, 0.283) |
| Sphere 80 | 487.47 (327.96, 609.55) | 1.004 (0.764, 1.436) | 0.348 (0.239, 0.696) | 0.023 (0.000, 0.368) |
| Torus | 473.90 (280.85, 602.22) | 5.835 (2.871, 13.074) | 5.089 (1.807, 9.688) | 0.062 (0.0001, 0.857) |
| Bunny | 449.16 (174.19, 591.39) | 11.131 (10.273, 11.821) | 9.862 (8.064, 11.960) | 0.894 (0.0320, 4.791) |
| Octopus | 401.94 (94.74, 554.11) | 12.564 (11.946, 13.040) | 11.409 (8.759, 14.415) | 0.251 (0.0001, 2.661) |
| Banana | 221.66 (27.17, 325.21) | 12.984 (12.50, 15.676) | 12.375 (11.880, 15.674) | 0.914 (0.0017, 4.976) |
| Liver | 147.01 (15.70, 241.02) | 12.825 (12.155, 13.350) | 10.487 (9.050, 12.928) | 1.319 (0.0004, 7.640) |
| Armadillo | 5.12 (0.79, 26.11) | 15.262 (14.018, 32.162) | 12.655 (9.956, 25.375) | 11.123 (0.0002, 72.419) |

## 표 3a. 변형 시간 통계 (ms)

| 모델 | 표준편차(SD) | 중앙값 | P95 | P99 |
|---|---|---|---|---|
| Sphere 20 | 0.327 | 0.811 | 1.066 | 1.583 |
| Sphere 80 | 0.151 | 1.389 | 1.739 | 1.984 |
| Torus | 2.703 | 4.885 | 9.053 | 9.297 |
| Bunny | 0.530 | 11.380 | 11.721 | 11.785 |
| Octopus | 0.356 | 12.735 | 13.007 | 13.030 |
| Banana | 0.392 | 13.090 | 13.358 | 13.582 |
| Liver | 0.435 | 13.067 | 13.286 | 13.335 |
| Armadillo | 2.294 | 14.609 | 17.664 | 24.805 |

## 표 3b. 충돌 시간 통계 (ms)

| 모델 | 표준편차(SD) | 중앙값 | P95 | P99 |
|---|---|---|---|---|
| Sphere 20 | 0.086 | 0.116 | 0.268 | 0.384 |
| Sphere 80 | 0.083 | 0.337 | 0.490 | 0.641 |
| Torus | 3.267 | 3.061 | 9.342 | 9.553 |
| Bunny | 1.042 | 9.554 | 11.631 | 11.915 |
| Octopus | 2.118 | 9.749 | 14.354 | 14.412 |
| Banana | 0.774 | 12.087 | 14.477 | 15.615 |
| Liver | 1.533 | 9.270 | 12.869 | 12.925 |
| Armadillo | 3.375 | 10.360 | 18.620 | 19.665 |

## 표 3c. 절단 시간 통계 (ms)

| 모델 | 표준편차(SD) | 중앙값 | P95 | P99 |
|---|---|---|---|---|
| Sphere 20 | 0.064 | 0.051 | 0.165 | 0.250 |
| Sphere 80 | 0.066 | 0.003 | 0.081 | 0.299 |
| Torus | 0.172 | 0.009 | 0.315 | 0.780 |
| Bunny | 1.205 | 0.351 | 3.462 | 4.455 |
| Octopus | 0.583 | 0.026 | 1.109 | 2.593 |
| Banana | 1.456 | 0.054 | 4.094 | 4.801 |
| Liver | 2.303 | 0.024 | 6.636 | 7.469 |
| Armadillo | 18.949 | 0.047 | 57.943 | 71.280 |

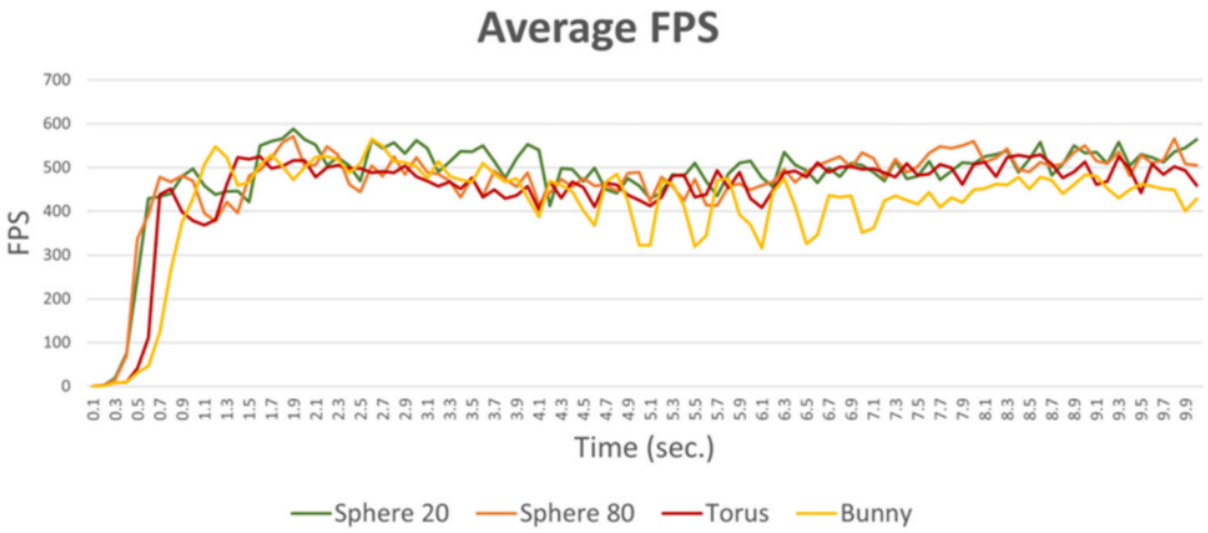



그림 4. Sphere 20, Sphere 80, Torus, Bunny의 평균 FPS. 가로축: 시뮬레이션 시간(s), 세로축: 평균 FPS.

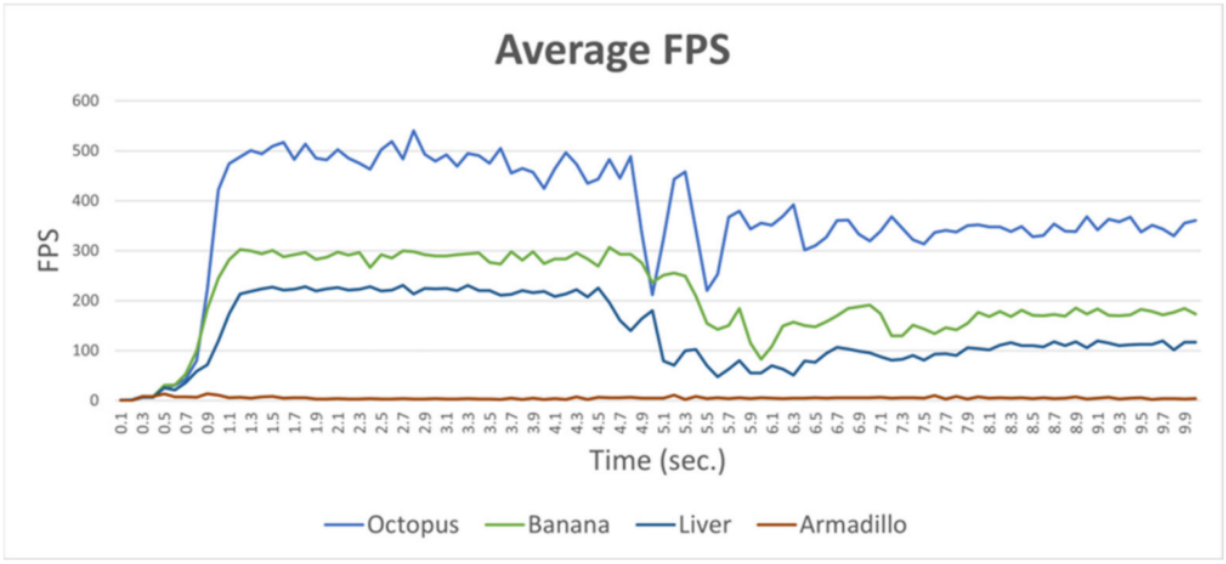



그림 5. Octopus, Banana, Liver, Armadillo의 평균 FPS. 가로축: 시뮬레이션 시간(s), 세로축: 평균 FPS.

8개 모델 모두에 대해 체적 보존을 평가했다. 초기 체적은 표면 삼각형에 발산 정리[25]를 적용하여 계산했다. 시뮬레이션 중 체적을 변형체 초기 체적의 백분율로 기록했다. 표 4에 체적 보존 통계를, 그림 7에 평균 체적 곡선을 제시한다.

모든 모델은 공통적으로 세 단계 반응을 보였다. 초기 정지 구간에서는 체적이 정확히 100%로 보존된다. 관측 구간 시작 후 3.5~6.9초 사이에 급격한 감소가 시작되며, 이는 절단 완료와 이후 분리 조각의 바닥 충돌에 맞물린다. 그 다음 안정적인 평탄 구간이 형성되어 관측 종료까지 유지된다.

이 반응의 크기에 따라 모델은 뚜렷한 두 집단으로 나뉘었으며 중간 사례는 없었다. 첫 집단인 Sphere 20, Sphere 80, Torus, Bunny는 사실상 전체 체적을 유지했고, 평균된 최솟값이 96.37% 아래로 떨어지지 않았다. Sphere 20이 전체적으로 가장 안정적이었다(평균 99.97%, SD 0.04%). 이러한 결과는 단순한 위상의 저·중해상도 메시에서 ISPC가 원래 체적을 효과적으로 복원하며, 토러스처럼 단순하지 않은 위상 자체가 체적 보존을 저해하지는 않음을 나타낸다.




그림 6 (앞부분). 왼쪽은 단계별 CPU·GPU 평균 시간(ms)의 시계열, 오른쪽은 히스토그램이다. (a) Sphere 20, (b) Sphere 80, (c) Torus.

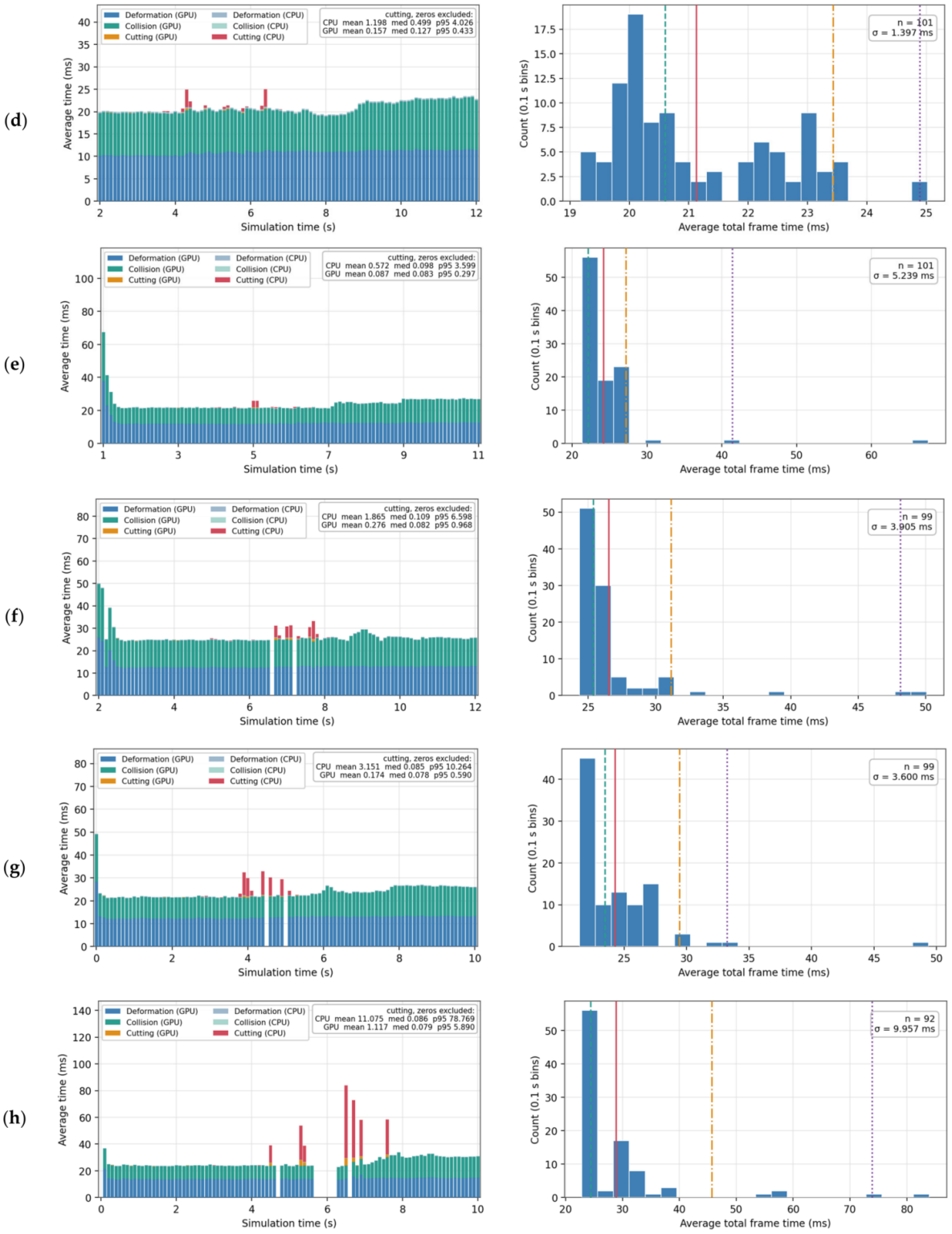



그림 6 (계속). (d) Bunny, (e) Octopus, (f) Banana, (g) Liver, (h) Armadillo. 빨간 실선=평균, 녹색 파선=중앙값, 노란 일점쇄선=95백분위수(P95), 보라색 점선=99백분위수(P99). 시계열: 가로축은 시뮬레이션 시간(s), 세로축은 평균 시간(ms). 히스토그램: 가로축은 평균 전체 프레임 시간(ms), 세로축은 구간별 개수.

두 번째 집단인 Liver, Banana, Armadillo, Octopus는 훨씬 큰 손실을 보였다. Liver, Banana, Armadillo는 비슷하게 움직여 최솟값 74.57~87.43% 이후 92.0~93.3%의 평탄 구간에 정착했다. 이 세 모델의 체적 중앙값은 100%로 유지되었다. 충돌 전 평탄 구간이 관측 구간의 절반 이상이며 체적 손실은 충돌 이후 구간에 한정됨을 의미한다.

Octopus는 다른 모든 모델보다 체적 보존이 크게 나빴다. 감소가 가장 일찍 시작되고, 약 27%로 가장 가파르게 줄었으며, 이후 전혀 회복하지 않았다. 평균된 체적은 73.43%에서 안정화되었다. 종합하면 기하 복잡도가 낮은 모델은 초기 체적의 0.4% 이내로 유지되지만 메시 복잡도가 커질수록 ISPC의 효과는 감소하며, 매우 상세한 Octopus가 한계 사례이다. 모델별 프레임 샘플 체적 백분율의 상세 히스토그램은 부록 A 그림 A2에 제시한다.

## 표 4. 10초 관측 구간의 체적 보존 통계 (초기 체적 대비 %)

| 모델 | 평균 | 중앙값 | P95 | P99 | SD | 최소 |
|---|---|---|---|---|---|---|
| Sphere 20 | 99.967 | 99.956 | 100 | 100 | 0.042 | 99.78 |
| Sphere 80 | 99.858 | 99.816 | 100 | 100 | 0.137 | 99.55 |
| Torus | 99.805 | 99.782 | 100 | 100 | 0.409 | 96.37 |
| Bunny | 99.791 | 99.825 | 100 | 100 | 0.239 | 98.94 |
| Octopus | 85.259 | 73.575 | 100 | 100 | 13.171 | 72.97 |
| Banana | 96.853 | 100 | 100 | 100 | 4.853 | 74.57 |
| Liver | 97.55 | 100 | 100 | 100 | 3.533 | 87.43 |
| Armadillo | 97.928 | 100 | 100 | 100 | 4.079 | 86.73 |

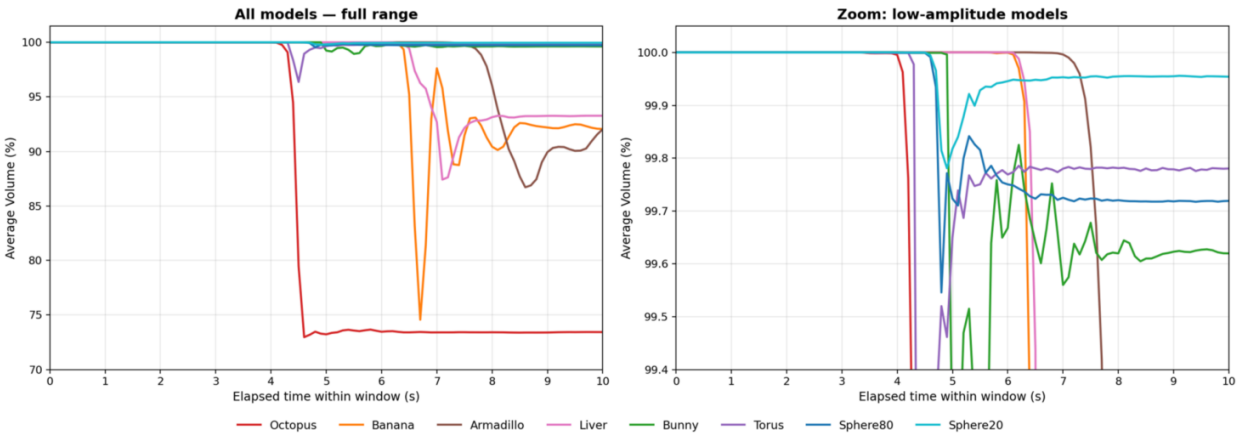



그림 7. Sphere 20, Sphere 80, Torus, Bunny, Octopus, Banana, Liver, Armadillo의 평균 체적. 가로축: 시뮬레이션 시간(s), 세로축: 초기 체적 대비 체적(%).

## 3.2. 다중 절단 실험

Sphere 80, Torus, Banana의 다중 절단 실험을 각각 그림 8a, 8b, 8c에 제시한다. 사용자가 도구를 자유롭게 움직여 각 모델에 여러 번 절단할 수 있었다. 절단 완료 후 중력을 활성화하여 분리된 조각을 바닥으로 떨어뜨렸다. 그림 8처럼 분리 조각은 바닥 및 서로와 자연스럽게 상호작용하고, 시뮬레이션 전반에서 사실적인 변형과 충돌 반응을 보였다.

자유로운 다중 절단의 계산 성능을 정량화하기 위해 각 모델을 독립적으로 5회 실행하고, 프레임별 단계 시간을 0.1초마다 샘플링했다. 엔진 초기 구동의 과도 현상을 제외하면서 전체 절단과 절단 후 상호작용을 포함하도록 분석 구간을 Sphere 80은 1.5~13.5초, Torus는 2~17초, Banana는 3~17초로 선택했다. 각각 평균 샘플 121개, 151개, 141개를 얻었다.

그림 9는 단계별 CPU·GPU 평균 시간(ms)의 시계열과 히스토그램을 보여준다. Sphere 80은 실험 내내 실시간 성능을 유지했고, 전체 프레임 시간 평균은 1.81ms(중앙값 1.42ms, SD 0.77ms), 꼬리 지연은 P95 3.14ms와 P99 3.90ms였다. Torus 평균은 4.14ms(중앙값 2.35ms, SD 3.85ms, P95 13.69ms, P99 14.15ms)였다. Banana 평균은 14.37ms(중앙값 10.31ms, SD 7.28ms, P95 26.52ms, P99 27.53ms)였다.

두 큰 모델은 평균이 중앙값보다 상당히 크다. 프레임 시간 분포가 두 봉우리를 갖기 때문이다. 안정적인 절단 전 상태 이후, 분리 조각이 중력으로 떨어지고 충돌 반응이 활성화되면서 별도의 절단 후 상태가 시작된다.

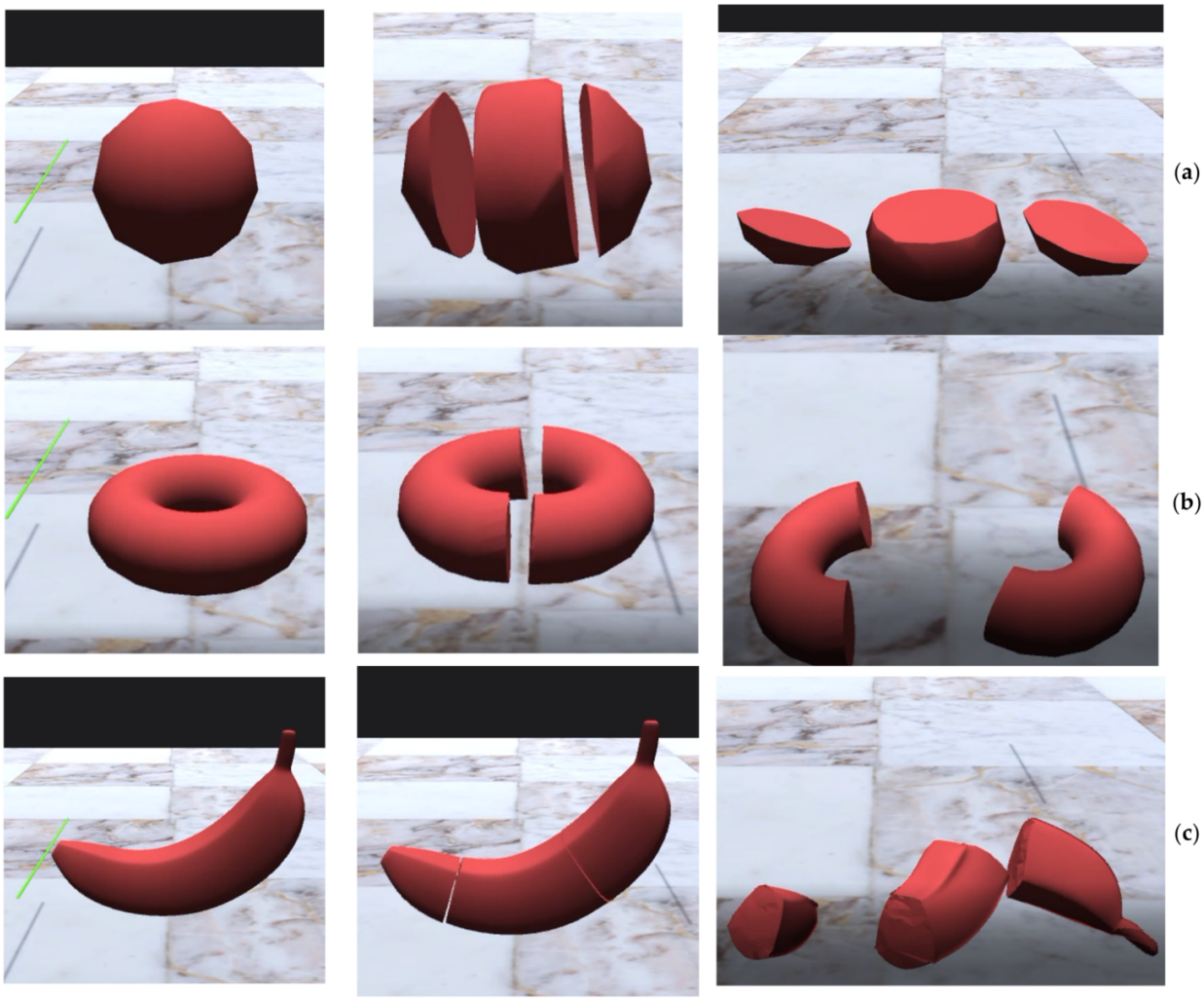



그림 8. 다중 절단. (a) Sphere 80, (b) Torus, (c) Banana. 녹색 선은 절단 도구의 경로를 나타낸다.

세 모델 모두 변형이 전체 비용을 지배했다. 변형 평균은 Sphere 80 1.57ms, Torus 3.18ms, Banana 10.64ms였다. 충돌 검출은 절단 전에는 비활성화되어 있었고 분리 후 두 번째로 큰 비용이 되었다. 충돌이 활성화된 프레임에서 세 모델의 충돌 평균은 각각 0.67ms, 4.32ms, 9.06ms였다. Torus와 Banana의 절단 후 충돌 비용은 변형과 같은 규모에 가까워졌고, 이는 두 번째 프레임 시간 상태로의 전환을 설명한다.

반면 절단 자체의 기여는 작았다. 절단 작업을 수행한 프레임에서 평균은 Sphere 80 0.023ms, Torus 0.028ms, Banana 0.456ms에 불과했다. 가장 큰 모델에서 전체 누적 프레임 시간의 약 1.3%에 해당한다. 이는 반복적인 자유 절단에서도 GPU 기반 절단 파이프라인이 프레임당 오버헤드를 거의 추가하지 않으며, 절단 후 전체 성능은 위상 변경보다 변형과 조각 간 충돌 처리에 주로 좌우됨을 나타낸다.

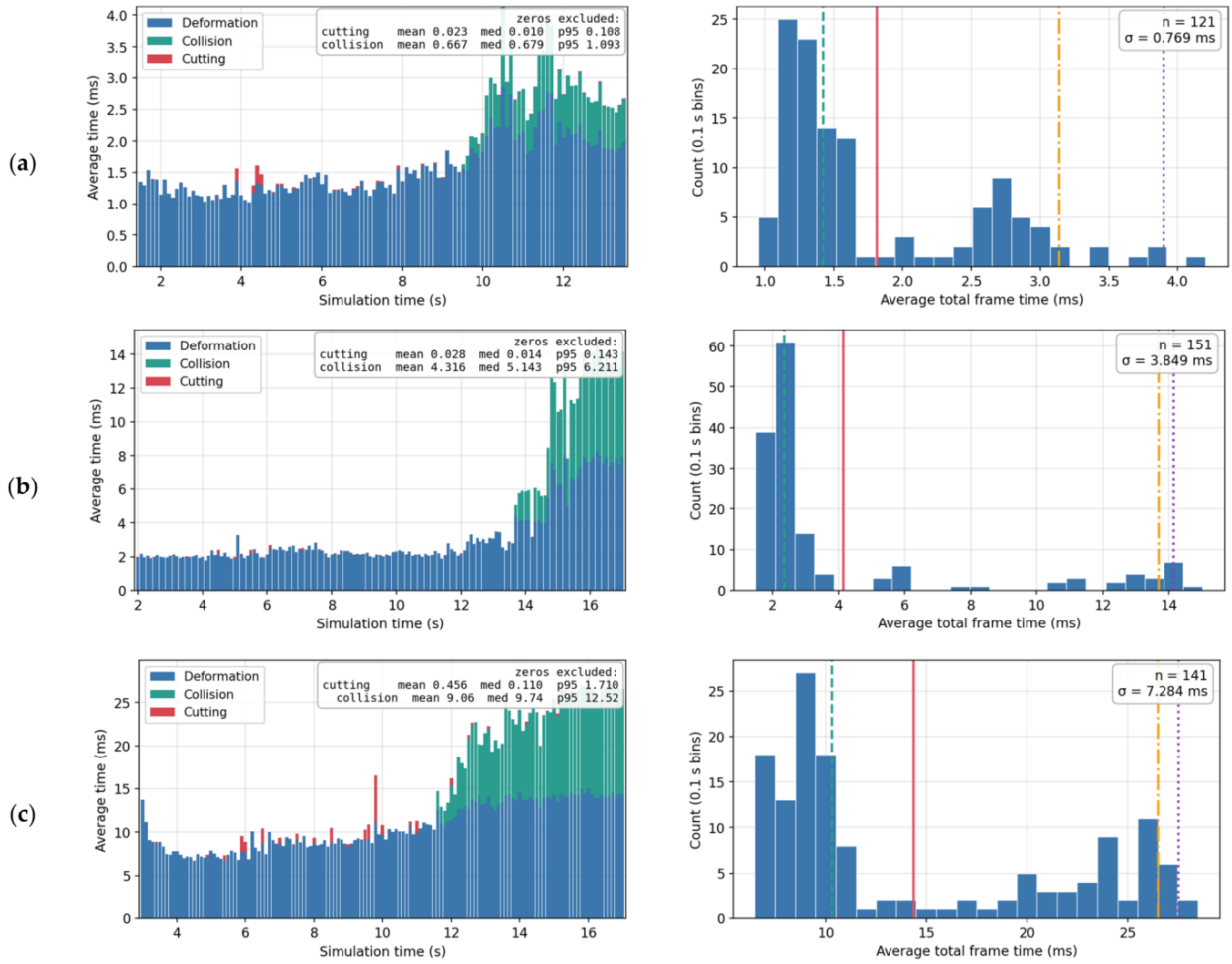



그림 9. 왼쪽은 단계별 CPU·GPU 평균 시간(ms)의 시계열, 오른쪽은 히스토그램이다. (a) Sphere 80, (b) Torus, (c) Banana. 빨간 실선=평균, 녹색 파선=중앙값, 노란 일점쇄선=P95, 보라색 점선=P99.

다중 절단의 메시 증가를 평가하기 위해 3개 모델에서 독립적인 5회 실행 동안 첫 번째와 두 번째 절단에서 생성된 정점·삼각형 수를 기록했다. Banana는 첫 절단에서 평균 새 정점 696개(SD 88), 삼각형 1493개(SD 176)를 생성했다. 원래 메시의 정점 2798개와 삼각형 5592개 대비 각각 24.9%, 26.7%이다. 두 번째 절단은 정점 853개(SD 83), 삼각형 1807개(SD 166)를 생성했으며 원래 수의 30.5%, 32.3%였다. 두 번 절단 후 정점은 55.4%, 삼각형은 59.0% 증가했다.

Torus도 비슷한 상대 증가를 보였다. 첫 절단은 정점 112개와 삼각형 262개를 추가했으며, 원래 정점 400개와 삼각형 800개의 28.0%, 32.8%였다. 두 번째는 정점 119개와 삼각형 276개(29.8%, 34.5%)를 추가하여 누적 증가율이 57.8%, 67.2%였다.

가장 작은 Sphere 80에서는 절단마다 원래 메시 전체보다 더 많은 새 요소를 생성했다. 첫 번째는 정점 52개와 삼각형 114개로 원래 42개와 80개의 123.8%, 142.5%였고, 두 번째는 정점 53개와 삼각형 122개로 126.2%, 152.8%였다. 따라서 두 번 절단하면 메시 크기가 대략 세 배가 되었다(정점 +250.5%, 삼각형 +294.8%).

이는 절단 경로의 교차 삼각형 수가 기하학적 최소치 이하로 줄어들 수 없어서, 절단의 상대적인 위상 비용이 메시 해상도에 반비례함을 반영한다. 모든 모델과 두 절단에서 새 삼각형 대 새 정점 비율은 약 2.1~2.3으로 안정적이었다. 이는 복제한 절단 경로 정점 하나가 각 면에서 약 두 개의 새 삼각형에 공유되는 그림 1의 세분화 방식과 일치한다. 이렇게 메시가 크게 증가했는데도 모든 모델에서 절단 단계의 평균 프레임 비용은 0.5ms 미만이었다.

## 3.3. 이전 연구와의 비교

GPU 기반 ISPC 방법, 이전에 발표한 슬라이싱 방법[26], CPU 기반 ISPC 방법[20]의 계산 성능을 비교했다. 표 5는 세 방법의 알고리즘 차이를 설명한다. 더 적절하게 비교하기 위해 미리 정의한 경로로 도구를 이동하여 이전 연구의 절단 경로를 가깝게 따랐다. 표 6은 세 방법의 평균 FPS를 요약한다. 마지막 두 열은 GPU ISPC 방법의 슬라이싱 및 CPU ISPC 대비 FPS 향상률(%)이다.

슬라이싱 대비 제안 GPU ISPC는 Sphere 20, Sphere 80, Bunny, Octopus, Banana에서 각각 133%, 175%, 462%, 677%, 590%의 FPS 향상을 달성했다. 유일한 예외는 Armadillo로, 슬라이싱 방법의 프레임률이 1.53% 더 높았다. CPU ISPC 대비 Sphere 20, Sphere 80, Octopus, Banana의 성능 향상은 각각 179%, 193%, 395%, 327%였다. Bunny와 Armadillo는 [20]에서 평가하지 않았으므로 이 비교에서 제외했고, Torus는 이전 두 연구 모두에 포함되지 않아 생략했다.

## 표 5. 비교한 세 방법의 알고리즘 차이

### 변형 방법

| 슬라이싱 [26] | CPU ISPC [20] | GPU ISPC |
|---|---|---|
| 거리·굽힘 제약을 갖는 GPU 기반 PBD. | 거리·굽힘·내부 형상 보존 제약을 갖는 GPU 기반 PBD. | 거리·굽힘·내부 형상 보존 제약을 갖는 GPU 기반 PBD. |

### 체적 보존

| 슬라이싱 [26] | CPU ISPC [20] | GPU ISPC |
|---|---|---|
| 체적 보존 없음. | ISPC 구현으로 체적 보존. 절단 후 ISPC를 재생성한다. | ISPC 구현으로 체적 보존. ISPC 재생성의 전수 탐색을 공간 격자 순회와 레이 마칭으로 대체하여 내부 연결 후보를 찾는다. |

### 절단 조각 간 충돌 처리

| 슬라이싱 [26] | CPU ISPC [20] | GPU ISPC |
|---|---|---|
| 없음. | GPU 충돌 처리의 넓은 단계에서 AABB를, 좁은 단계에서 M ller-Trumbore 알고리즘을 사용한다. 속도 기반 충돌 반응을 적용한다. | GPU에서 공간 해시를 사용한 정점-삼각형 충돌을 구현한다. 각 삼각형은 주변 3×3×3 셀에서 후보를 모으고 위상적 이웃은 제외한다. |

## 절단

| 슬라이싱 [26] | CPU ISPC [20] | GPU ISPC |
|---|---|---|
| 정적인 평면으로 절단한다. 교차 삼각형을 제거하고 경계를 재구축하며 새 삼각형으로 절단 조각을 덮고 세분화한다. 완전한 단일 슬라이싱만 지원한다. 알고리즘 전체가 CPU에서 실행된다. | 움직이는 도구로 점진적 절단을 구현한다. 정확한 교차 경로를 따라 삼각형을 세분화하고 절단 경계 정점을 복제하여 조각을 분리한다. 알고리즘 전체가 CPU에서 실행된다. | CPU ISPC와 비슷하지만 삼각형-삼각형 교차 검사는 GPU에서 수행한다. 절단 없음·시작·중·종료의 4상태 머신을 구현한다. 절단 시작과 절단 중에 절단 연산을 수행하고, 구멍 채우기·제약 재구축·버퍼 동기화는 절단 종료까지 미룬다. |

## 교차 검출

| 슬라이싱 [26] | CPU ISPC [20] | GPU ISPC |
|---|---|---|
| 전수 탐색. 각 표면 삼각형을 쓸기 평면의 두 삼각형과 삼각형-모서리 검사로 비교한다. | 넓은 단계에서 OBB를 사용하고 좁은 단계에서는 미리 계산한 삼각형 인접성을 BFS로 순회한다. | GPU에서 병렬 전수 삼각형-모서리 검사를 수행한다. 교차 결과를 지정 버퍼에 누적하고 CPU로 읽어온다. |

## 렌더링 방법

| 슬라이싱 [26] | CPU ISPC [20] | GPU ISPC |
|---|---|---|
| Unity 내장 Mesh Renderer를 사용한다. 매 프레임 갱신된 정점 위치를 GPU에서 읽어와야 한다. | Unity 내장 Mesh Renderer를 사용한다. 매 프레임 갱신된 정점 위치를 GPU에서 읽어와야 한다. | Unity의 DrawProcedural()로 GPU 기하 버퍼에서 모델을 직접 렌더링한다. 따라서 GPU와 CPU 간 데이터 전송이 필요하지 않다. |

## 표 6. CPU ISPC와 GPU ISPC 방법의 성능 평가

| 모델 | 슬라이싱 [26] 평균 FPS | CPU ISPC [20] 평균 FPS | GPU ISPC 평균 FPS | 슬라이싱 대비 향상률 (%) | CPU ISPC 대비 향상률 (%) |
|---|---|---|---|---|---|
| Sphere 20 | 205.9 | 172.1 | 481.04 | 133 | 179 |
| Sphere 80 | 177.2 | 166.1 | 487.47 | 175 | 193 |
| Bunny | 79.9 | - | 449.16 | 462 | - |
| Octopus | 51.7 | 81.1 | 401.94 | 677 | 395 |
| Banana | 32.1 | 51.8 | 221.66 | 590 | 327 |
| Armadillo | 5.2 | - | 5.12 | −1.53 | - |

# 4. 논의

이 연구는 ISPC로 강화한 표면 메시에 기반한 실시간 절단 시뮬레이션 프레임워크를 제시했다. 해상도와 위상이 다른 8개 모델에서 단일 절단, 다중 절단, 체적 보존 실험으로 평가했다. 결과는 상호작용 가능한 프레임률에서 안정적이고 시각적으로 사실적인 절단을 보여준다. ISPC는 중력과 충돌이 유발하는 큰 변형에서 체적과 구조적 일체성을 보존한다.

GPU 구현은 이전 접근법보다 계산 성능을 크게 개선했다. 삼각형-삼각형 교차 단계를 병렬화하고 GPU 상주 기하 버퍼에서 직접 렌더링하여, 슬라이싱 대비 최대 677%, CPU ISPC 대비 최대 395%의 프레임률 향상을 얻었다. 특히 절단 단계 자체는 더 이상 병목이 아니다. 두 번 절단 후 메시가 55~295% 증가했음에도 다중 절단 실험에서 절단 평균 비용은 0.5ms 미만이었다. 가장 큰 모델에서도 누적 프레임 시간의 약 1.3%이다.

정상 상태 성능은 대신 변형과 조각 간 충돌 처리에 좌우된다. 두 단계는 Armadillo를 제외한 모든 모델에서 프레임 시간의 95% 이상을 차지한다. Torus는 이러한 상태 의존성을 가장 분명하게 보여준다. 분리 조각이 지속적인 접촉에 들어가면 전체 비용이 대략 세 배가 되어 큰 모델에서 관찰한 두 봉우리의 프레임 시간 분포를 만든다.

Armadillo는 현재의 확장 한계를 두 방식으로 드러낸다. 평균 프레임 비용은 정점·삼각형 수에 따라 증가하는 변형과 충돌 단계가 지배한다. 동시에 프레임 시간 변동은 꼬리가 극도로 두꺼운 절단 분포에 의해 좌우된다. 절단 중앙값은 매우 작지만, CPU에서 절단 틈 삼각분할과 제약 재구축을 수행하는 드문 마무리 프레임이 최악의 지연을 일으켜 상호작용 가능한 프레임률 아래로 떨어뜨린다.

따라서 추가 개선에는 고해상도 메시의 PBD 솔버와 자기 충돌 단계 최적화, 그리고 절단 마무리를 주요 실행 경로에서 분리하는 작업이 모두 필요하다. 예를 들면 비동기 또는 점진적인 제약 재구축을 사용할 수 있다.

체적 보존도 해상도와 기하에 의존한다. 단순한 위상의 Sphere 20, Sphere 80, Torus, Bunny는 0.4% 이내의 차이로 체적을 유지하므로, 단순하지 않은 위상 자체가 보존을 저해하지 않음을 확인한다. Liver, Banana, Armadillo는 충돌 후 92.0~93.3% 평탄 구간에 정착한다. Octopus는 한계 사례로, 제약 강성을 1까지 높였는데도 평균 체적이 약 27% 감소하고 회복 없이 73.43%에 안정화된다(평균된 최솟값 72.97%).

우리는 이를 ISPC 생성 전략 때문이라고 본다. 촉수처럼 얇고 곡률이 큰 부분에서 내부 방향으로 광선을 발사하면 압축에 저항하지 못하는 짧거나 부적절한 방향의 내부 제약이 만들어진다. 따라서 복잡한 형상에는 기하 특성을 고려한 제약 배치가 필요하다.

프레임워크는 Unity가 Direct3D, Vulkan, Metal 백엔드에서 제공하는 표준 컴퓨트 셰이더 기능, 즉 구조화 버퍼, 스레드 그룹 공유 메모리, 원자 연산에만 의존한다. 따라서 이 API가 지원하는 GPU 아키텍처라면 알고리즘을 변경하지 않고 이식할 수 있다. 변형·충돌·교차 단계의 성능은 주로 사용 가능한 연산 유닛 수에 따라 달라지고, CPU에서 실행되는 절단 마무리는 GPU 아키텍처와 무관하다.

메모리 요구량은 초기화 때 불러온 메시 크기의 일정 배수를 예약하는 버퍼 사전 할당 정책으로 정해진다. VRAM이 작은 GPU에서는 사전 할당 배수를 줄일 수 있지만 지원 가능한 연속 절단 횟수의 한도가 낮아진다.

향후 연구는 기하학적으로 복잡한 물체의 ISPC 생성 개선, 고해상도 메시의 변형·자기 충돌 최적화, 수술 시뮬레이션을 위한 더 사실적인 재료 특성 도입, 시뮬레이션 충실도 검증에 집중할 것이다. 마지막으로 동적 메시 자료구조의 복잡성 때문에 절단 단계는 부분적으로만 병렬화되어 있으므로 완전한 GPU 기반 절단 알고리즘은 중요한 연구 방향으로 남는다.

# 저자 기여 및 선언

저자 기여: 개념 수립, L.K., Y.-J.C., M.H.; 방법론, L.K., Y.-J.C., M.H.; 소프트웨어, L.K.; 검증, L.K., M.H.; 형식적 분석, Y.-J.C.; 조사, Y.-J.C., L.K.; 자원, L.K.; 데이터 정리, M.H.; 초안 작성, L.K.; 검토와 편집, L.K., M.H.; 시각화, L.K.; 지도, M.H.; 프로젝트 관리, M.H.; 연구비 확보, M.H. 모든 저자는 출판된 원고를 읽고 동의했다.

연구비: 이 연구는 교육부 재원으로 한국연구재단(NRF)을 통한 기초연구사업(NRF-2022R1I1A3069371), BK21 FOUR(대학원 혁신 및 우수 연구대학 육성, 번호 5199990914048), 순천향대학교 연구비의 지원을 받았다.

데이터 가용성: 이 연구의 독창적인 기여는 논문에 포함되어 있다. 추가 문의는 교신저자에게 할 수 있다.

이해충돌: 저자들은 이해충돌이 없음을 선언한다.

저작권 및 라이선스: © 2026 저자. 라이선스 제공자 MDPI, Basel, Switzerland. 이 논문은 Creative Commons Attribution(CC BY) 라이선스 조건으로 배포되는 공개 접근 논문이다.

# 부록 A

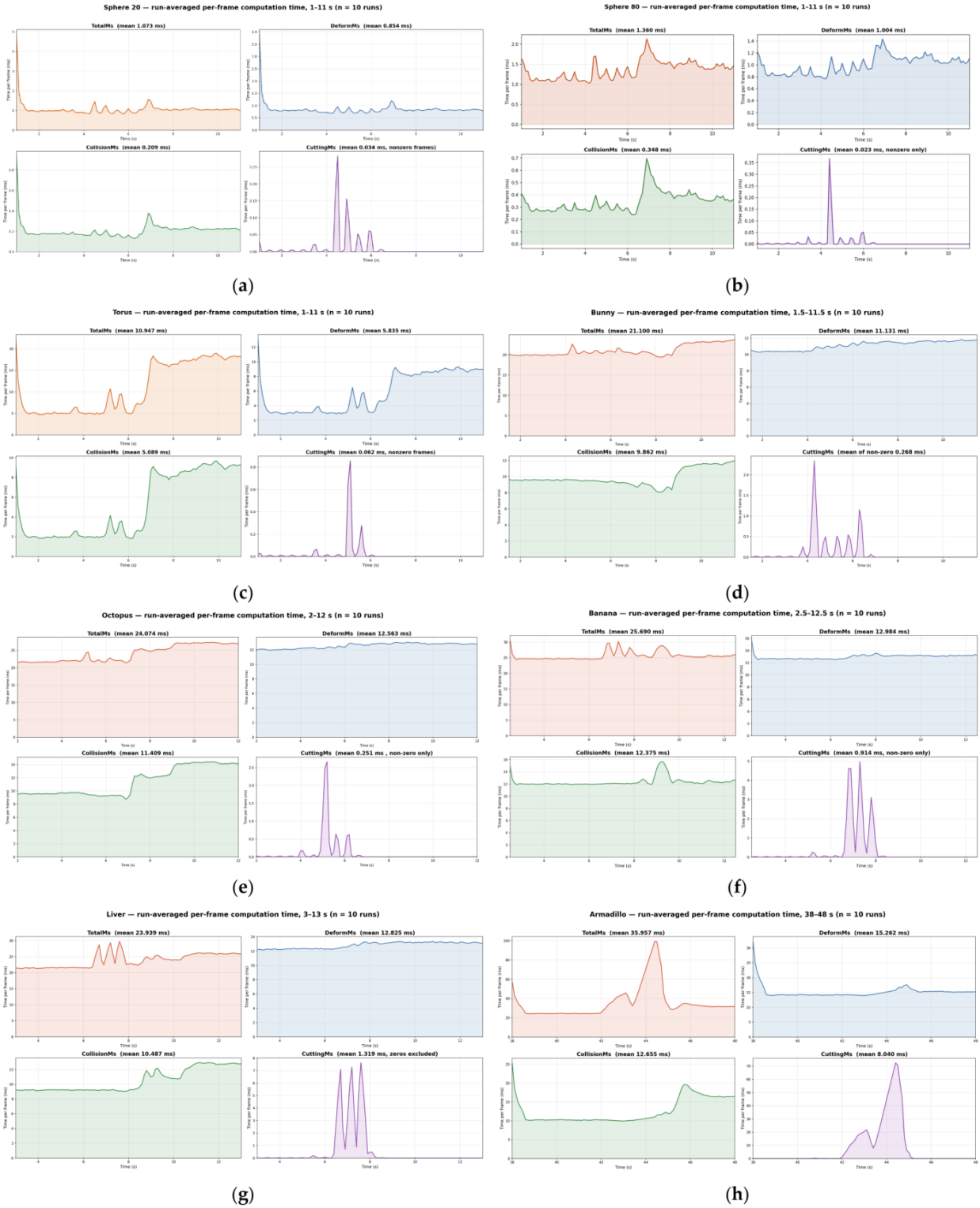



그림 A1. 샘플링한 프레임당 평균 전체 시뮬레이션, 변형, 충돌, 절단 시간. (a) Sphere 20, (b) Sphere 80, (c) Torus, (d) Bunny, (e) Octopus, (f) Banana, (g) Liver, (h) Armadillo. 각 패널: Total=전체, Deformation=변형, Collision=충돌, Cutting=절단. 가로축: 시뮬레이션 시간(s), 세로축: 시간(ms).

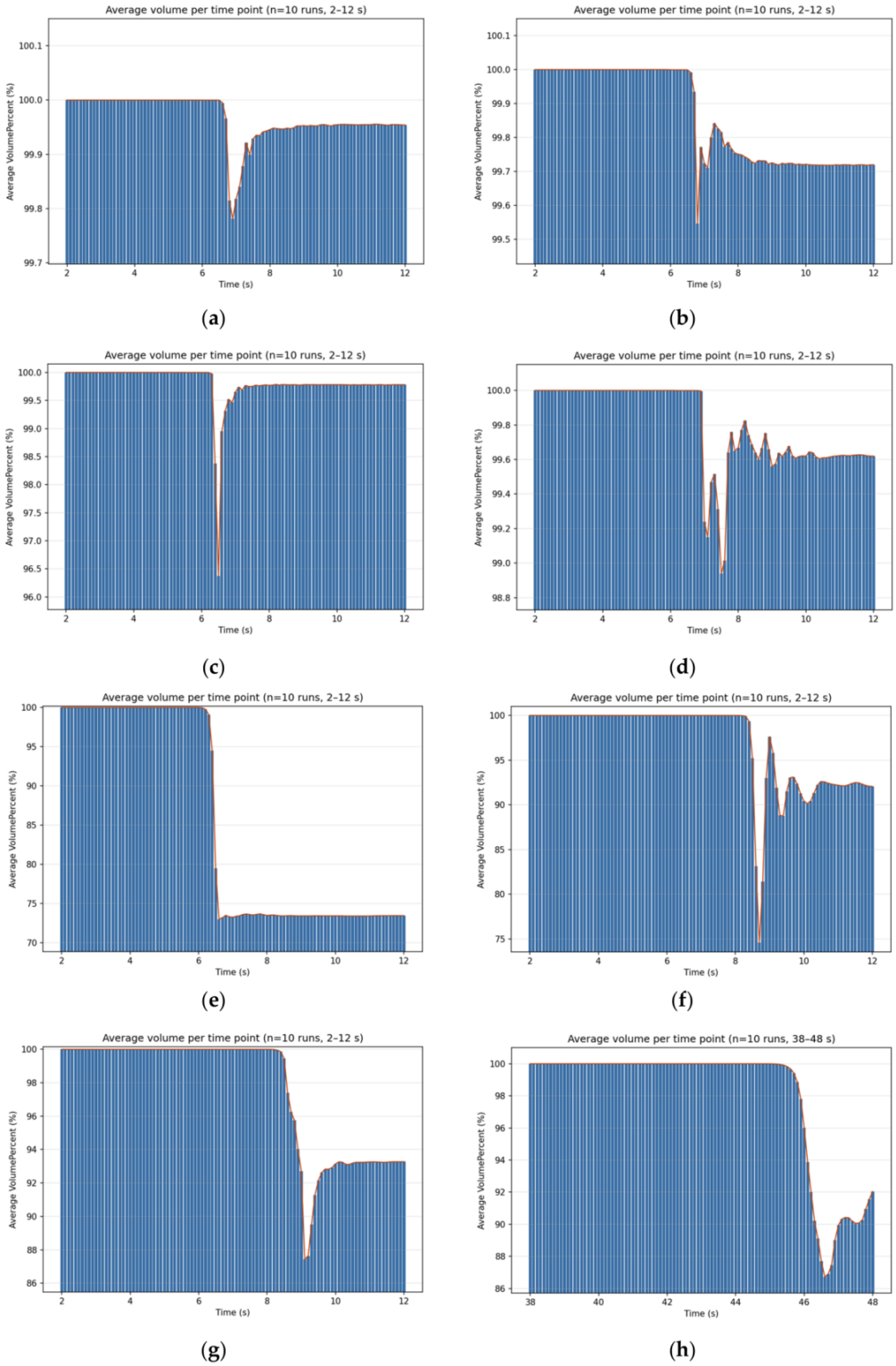



그림 A2. 프레임당 평균 체적 히스토그램. (a) Sphere 20, (b) Sphere 80, (c) Torus, (d) Bunny, (e) Octopus, (f) Banana, (g) Liver, (h) Armadillo. 가로축: 초기 체적 대비 체적(%), 세로축: 구간별 개수.

# 참고문헌

검색과 원문 대조를 위해 서지 정보는 원문 표기를 유지한다.


1. Portelli, M.; Bianco, S.F.; Bezzina, T.; Abela, J.E. Virtual reality training compared with apprenticeship training in laparoscopic surgery: A meta-analysis. Ann. R. Coll. Surg. Engl. 2020, 102, 672–684.

2. Stepan, K.; Zeiger, J.; Hanchuk, S.; Del Signore, A.; Shrivastava, R.; Govindaraj, S.; Iloreta, A. Immersive virtual reality as a teaching tool for neuroanatomy. Int. Forum Allergy Rhinol. 2017, 7, 1006–1013.

3. Imran, E.; Adanir, N.; Khurshid, Z. Significance of Haptic and Virtual Reality Simulation (VRS) in the Dental Education: A Review of Literature. Appl. Sci. 2021, 11, 10196.

4. Riddle, E.W.; Kewalramani, D.; Narayan, M.; Jones, D.B. Surgical Simulation: Virtual Reality to Artificial Intelligence. Curr. Probl. Surg. 2024, 61, 101625.

5. Wu, J.; Westermann, R.; Dick, C. A Survey of Physically Based Simulation of Cuts in Deformable Bodies. Comput. Graph. Forum 2015, 34, 161–187.

6. Nienhuys, H.W.; Frank van der Stappen, A. A Surgery Simulation Supporting Cuts and Finite Element Deformation. In Proceedings of the Medical Image Computing and Computer-Assisted Intervention—MICCAI 2001, Utrecht, The Netherlands, 14–17 October 2001.

7. Bielser, D.; Glardon, P.; Teschner, M.; Gross, M. A state machine for real-time cutting of tetrahedral meshes. Graph. Model. 2004, 66, 398–417.

8. Steinemann, D.; Otaduy, M.A.; Gross, M. Fast arbitrary splitting of deforming objects. In Proceedings of the 2006 ACM SIG- GRAPH/Eurographics Symposium on Computer Animation (SCA ’06); Eurographics Association: Goslar, Germany, 2006; pp. 63–72.

9. Steinemann, D.; Harders, M.; Gross, M.; Szekely, G. Hybrid Cutting of Deformable Solids. In Proceedings of the IEEE Conference on Virtual Reality (VR ’06); IEEE Computer Society: Los Alamitos, CA, USA, 2006; pp. 35–42.

10. Jia, S.; Zhang, W.; Yu, X.; Pan, Z. CPU–GPU Parallel Framework for Real-Time Interactive Cutting of Adaptive Octree-Based Deformable Objects. Comput. Graph. Forum 2018, 37, 45–59.

11. Kim, J.-H.; Lee, J. Advanced GPU Techniques for Dynamic Remeshing and Self-Collision Handling in Real-Time Cloth Tearing. IEEE Access 2025, 13, 3188–3203.

12. Byeon, S.P.; Lee, D.Y. Adaptive surface representation based on homogeneous hexahedrons for interactive simulation of soft tissue cutting. Comput. Methods Programs Biomed. 2021, 200, 105873.

13. Lee, D.-K.; Kim, T.-W.; Choi, Y.-J.; Hong, M. Volumetric Object Modeling Using Internal Shape Preserving Constraint in Unity 3D. Intell. Autom. Soft Comput. 2022, 32, 1541–1556.

14. Bender, J.; Müller, M.; Macklin, M. A survey on position based dynamics. In Proceedings of the European Association for Computer Graphics: Tutorials (EG ’17); Eurographics Association: Goslar, Germany, 2017; pp. 1–31.

15. Müller, M.; Heidelberger, B.; Hennix, M.; Ratcliff, J. Position based dynamics. J. Vis. Comun. Image Represent. 2007, 18, 109–118.

16. Wu, H.; Ye, F.; Gao, Y.; Cong, Y.; Hao, A. Real-Time Laparoscopic Cholecystectomy Simulation Using a Particle-Based Physical System. Complex Syst. Model. Simul. 2022, 2, 186–196.

17. Huang, T.; Zhang, N.; Zhang, X.; Liao, H. Anatomy Education Method using Autostereoscopic 3D Image Overlay and Mid-Air Augmented Reality Interaction. SID Symp. Dig. Tech. Pap. 2022, 53, 91–94.

18. Yu, P.; Zhao, Z.; Wang, R.; Pan, J. Real-time soft body dissection simulation with parallelized graph-based shape matching on GPU. Comput. Methods Programs Biomed. 2024, 250, 108171.

19. Haiderbhai, M.; Gondokaryono, R.; Wu, A.; Kahrs, L.A. Sim2Real Rope Cutting With a Surgical Robot Using Vision-Based Reinforcement Learning. IEEE Trans. Autom. Sci. Eng. 2025, 22, 4354–4365.

20. Khan, L.D.; Kim, T.; Hong, M. Real-Time Progressive Cutting of Deformable Objects in Unity 3D with Internal Shape Preserving Constraints. Appl. Sci. 2025, 15, 12984.

21. Va, H.; Choi, M.-H.; Hong, M. Real-Time Cloth Simulation Using Compute Shader in Unity3D for AR/VR Contents. Appl. Sci. 2021, 11, 8255.

22. Teschner, M.; Heidelberger, B.; Müller, M.; Pomerantes, D.; Gross, M.H. Optimized Spatial Hashing for Collision De- tection of Deformable Objects. In Proceedings of the International Symposium on Vision, Modeling, and Visualiza- tion, Munich, Germany, 19–21 November 2003. Available online: https://cg.informatik.uni-freiburg.de/publications/ 2003_VMV_collisionDetectionHashing.pdf (accessed on 16 June 2026).

23. Meisters, G.H. Polygons Have Ears. Am. Math. Mon. 1975, 82, 648–651.

24. Shewchuk, J.R. Delaunay refinement algorithms for triangular mesh generation. Comput. Geom. 2002, 22, 21–74.

25. Lancaster, J.L.; Eberly, D.; Alyassin, A.; Downs, J.H., III; Fox, P.T. A geometric model for measurement of surface distance, surface area, and volume from tomographic images. Med. Phys. 1992, 19, 419–431.

26. Khan, L.; Choi, Y.-J.; Hong, M. Cutting Simulation in Unity 3D Using Position Based Dynamics with Various Refinement Levels. Electronics 2022, 11, 2139.

## 출판사 고지

모든 출판물에 포함된 진술, 의견, 데이터는 전적으로 개별 저자와 기여자의 것이며 MDPI 및 편집자의 것이 아니다. MDPI 및 편집자는 본문에 언급된 아이디어, 방법, 지침, 제품으로 인해 사람이나 재산에 발생한 손해에 대한 책임을 부인한다.